Info: Only One F-Literal Expands

Document Number:	D0000R0
Date:	2026-04-08
Audience:	EWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

When two f-literals appear in the same call expression, one of them must lose.

P3412R3[1] proposes string interpolation for C++ through f-literals that expand to `__format__` function calls. When multiple f-literals appear as arguments to the same function, only the last is eligible for argument-list expansion - the rest collapse to `std::string`. This paper examines the structural consequences of that rule. What does it mean for structured logging, multi-field diagnostics, and variadic APIs? Can implicit conversions recover the lost structure? What alternatives exist in the design space?

Revision History

R0: April 2026

- Initial version.

---

1. Disclosure

The author provides information and serves at the pleasure of the committee.

The author maintains Boost.Beast, Boost.Http.Proto, Boost.Http.Io, and related networking libraries. The author has no competing string interpolation proposal and no direct stake in P3412R3[1]'s outcome.

This paper examines one structural property of the expansion model. It does not assess the overall quality of the proposal, which achieves its stated goals for the common case and has strong implementation evidence.

The analysis is drawn entirely from the published text of P3412R3[1] (Gustafsson, Zverovich, 2025-12-14).

This paper asks for nothing.

---

2. What P3412R3 Achieves

P3412R3[1] provides a string interpolation mechanism that:

- Operates in the early phases of translation, enabling macro expansion inside expression-fields and allowing syntax-coloring editors to participate without a full C++ parser
- Has been implemented in both a standalone pre-preprocessor (approximately 30 hours of work) and a Clang fork (approximately 50 hours), with the latter accumulating over 6000 compiles on Compiler Explorer in the first half of 2025
- Has received implementability feedback from four major compiler vendors (Clang, EDG, GCC, MSVC), none of whom identified fundamental obstacles

The design serves the common case cleanly:

```
std::string s = f"Value: {x}";
std::print(f"Value: {x}");
```

The first line calls `__format__` and receives a `std::string`. The second triggers the overload resolution expansion rule and passes the format string and arguments directly to `std::print`, avoiding the intermediate string. These two uses - producing a string and printing a string - are the primary use cases for string interpolation.

---

3. The Expansion Mechanism

P3412R3[1] Section 16 introduces a special overload resolution rule. When a function call contains a `__format__` call as an argument, and overload resolution either fails or requires a consteval constructor in the implicit conversion sequence, the last `__format__` call is expanded to its arguments and overload resolution is retried.

The transformation for the `std::print` case:

```
std::print(f"Value: {x}");

// After phase 6:
std::print(__format__("Value: {}", (x)));

// After expansion:
std::print("Value: {}", (x));
```

The expansion flattens the `__format__` call's argument list into the enclosing call's argument list. The format string becomes a positional argument. The expression-field values become subsequent positional arguments. The `__format__` wrapper disappears.

This is a positional model. The expanded arguments occupy fixed positions in the enclosing function's parameter list.

---

4. The Constraint

P3412R3[1] Section 16.1 states:

> *It should be allowed to have more than one f-literal in an argument list, but in this case only the last f-literal is considered for expansion.*

The rationale given in the same section:

> *The rationale for this is that in the normal formatting case that expansion of a f-literal results in any number of arguments and thus the matching parameter must be a pack, which is only allowed last in a parameter list.*

Expanding an f-literal produces a variable number of arguments - one format string plus N expression-field values. A parameter pack must be the last parameter. If the first f-literal in an argument list were expanded, the arguments it produces would need to be followed by whatever comes after, but the receiving function cannot declare a parameter list with a pack in the middle. The second f-literal cannot also expand because there is no second pack to receive its arguments.

The parameter pack must be last. Therefore the expansion must be last. Therefore only one.

---

5. What the Constraint Forecloses

5.1 Structured Logging

A structured logging system records fields as key-value pairs rather than as a single formatted string. Each field carries its format string and arguments separately so the backend can format lazily, filter by field, or serialize to structured formats like JSON.

```
log_structured(
    f"user={user}",
    f"action={action}",
    f"latency_ms={elapsed.count()}"
);
```

Under the expansion rule, only the last f-literal expands. The first two collapse to `std::string` before the function sees them. The logging backend receives two pre-formatted strings and one format-string-plus-arguments bundle. The structure of two out of three fields is gone before the backend can act on it.

5.2 Multi-Field Diagnostics

A diagnostic system that accepts multiple interpolated fields and formats each according to its own rules:

```
diagnostic(severity::warning,
    f"expected {expected_type}",
    f"got {actual_type}",
    f"at {location}"
);
```

Two fields arrive as pre-formatted strings. One arrives with its format string and arguments intact. The diagnostic system cannot apply uniform formatting to all three fields.

5.3 Variadic Composition

A generic function template that accepts any number of f-literals and processes each one independently:

```
template<typename... Fields>
void emit(Fields&&... fields);

emit(f"a={a}", f"b={b}", f"c={c}");
```

After phase 6 this becomes:

```
emit(
    __format__("a={}", (a)),
    __format__("b={}", (b)),
    __format__("c={}", (c))
);
```

Only the third `__format__` is eligible for expansion. The function receives two `std::string` values and one expanded format-string-plus-arguments sequence. There is no mechanism for a variadic template to receive multiple f-literals in their structured form.

Three f-literals. One expanded. Two collapsed.

---

6. The Implicit Conversion Workaround

One can define a type that captures the formatted result through implicit conversion from `std::string`:

```
struct log_field {
    std::string formatted;
    log_field(std::string s)
        : formatted(std::move(s)) {}
};

void log_structured(
    std::initializer_list<log_field> fields);
```

Each `__format__` call returns a `std::string`, which converts to `log_field`. This captures the formatted output.

What the workaround provides:

- The formatted string (the final output)

What the workaround does not provide:

- The format string (for deferred formatting)
- The individual arguments (for structured serialization)
- The ability to skip formatting entirely (for log-level filtering)
- Compile-time format string validation per field

The workaround also does not extend to variadic templates. A variadic function template deduces each argument's type individually. A collapsed `__format__` call arrives as `std::string`. An expanded one arrives as a format string followed by N arguments of heterogeneous types. A variadic template cannot distinguish between a `std::string` that was always a `std::string` and a `std::string` that was once an f-literal whose structure was discarded.

The workaround recovers the output. It does not recover the structure.

---

7. The Design Space

The positional expansion model - flattening `__format__` arguments into the enclosing call's argument list - is one point in the design space. The model has genuine advantages: it requires no new types, it introduces no lifetime concerns from captured references, and it composes naturally with existing functions like `std::print` that already accept a format string followed by arguments. The constraint examined in this paper is the cost of those advantages.

Other points in the design space exist. Each addresses the multi-expansion case differently and bears different costs.

7.1 Structured Return Type

If `__format__` returned a type that carried the format string and arguments as a bundle, multiple f-literals could coexist in a call without expansion:

```
auto bundle = f"Value: {x}";
// bundle carries ("Value: {}", x) as a typed object
```

Functions could accept this bundle type directly and extract the components. Multiple bundles could appear in the same argument list without competing for a single expansion slot.

The cost: the bundle must capture arguments by reference or by value. Capture by reference introduces lifetime questions - what happens when the expression-field value is a temporary? Capture by value introduces copies. Both require a new type in the standard library or the language.

7.2 Expansion of All F-Literals

If the expansion rule applied to every f-literal in the argument list, multiple expansions would produce a flat argument list with interleaved format strings and arguments. The receiving function would need to demultiplex at compile time - separating which arguments belong to which f-literal.

The cost: C++ parameter lists have no built-in delimiter between groups of arguments. The receiving function would need template metaprogramming to infer the boundaries, which is fragile and opaque. This approach also requires either multiple parameter packs (not supported) or a single pack with compile-time group parsing.

7.3 Separate Syntax for Structured F-Literals

The default f-literal could continue to produce `std::string` via `__format__`, while a separate prefix or wrapper produces a structured bundle:

```
auto s = f"Value: {x}";     // std::string
auto b = F"Value: {x}";     // structured bundle (hypothetical)

log_structured(F"a={a}", F"b={b}");  // both structured
```

The cost: a second string literal prefix and the associated teaching burden. Two prefixes that look similar but produce different types is a source of confusion, and distinguishing when to use which adds to the learning curve.

Each alternative trades the positional model's simplicity for the ability to carry structure through multiple f-literals in a single call.

---

8. Summary

P3412R3's expansion model serves the two most common string interpolation use cases - assigning to a string and passing to a formatting function - with a clean, well-implemented design.

Use Case	Supported	Mechanism
Single f-literal assigned to string	Yes	`__format__` returns `std::string`
Single f-literal passed to `std::print`	Yes	Overload resolution expansion
Multiple f-literals, each pre-formatted	Yes	Each `__format__` returns `std::string`
Multiple f-literals, each retaining structure	No	Only last expands; rest collapse
Variadic template accepting N structured f-literals	No	No multi-expansion mechanism

The constraint follows from one design choice: expanding f-literals by flattening their arguments into the enclosing call's positional argument list. A model that preserves f-literal structure as a first-class object would not have this limitation but would bear the costs documented in Section 7.

Acknowledgments

The structural concern examined in this paper was raised by a committee member during informal discussion of P3412R3[1], who identified the multi-f-literal case and the variadic limitation as primary motivations for voting against the proposal.

References

- [1] P3412R3: "String interpolation." Gustafsson, Zverovich. 2025-12-14. https://wg21.link/p3412r3